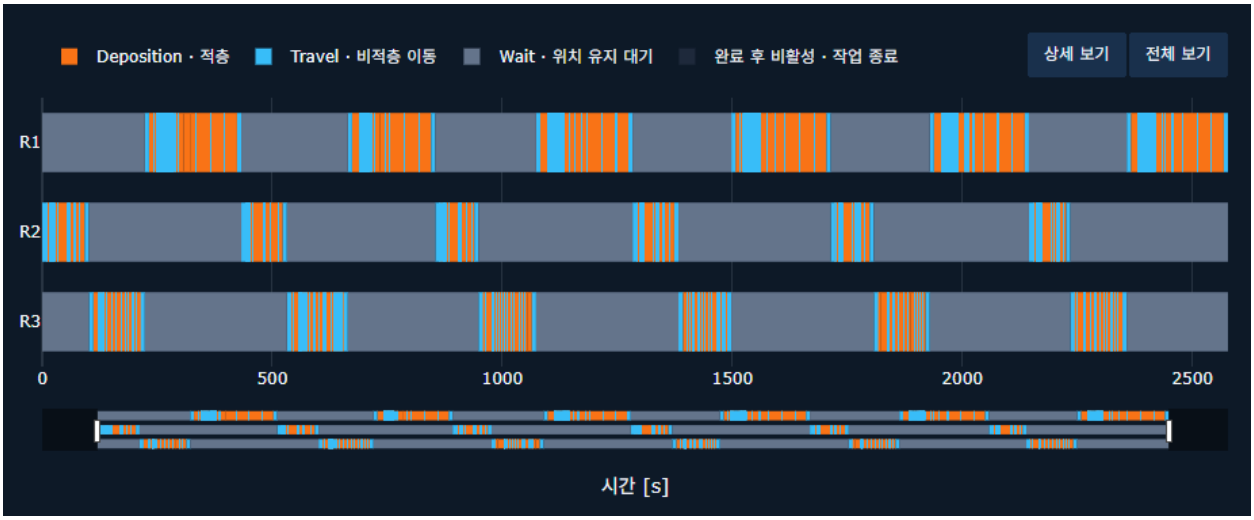
| d 름 | | **학번** | **2022089761** |
|---|---|---|---|
| **깃허브 링크** | | | |

- 알고리즘 1 번 | 1.0.2 Main Layer PPO
  - 모델별 적용 결과

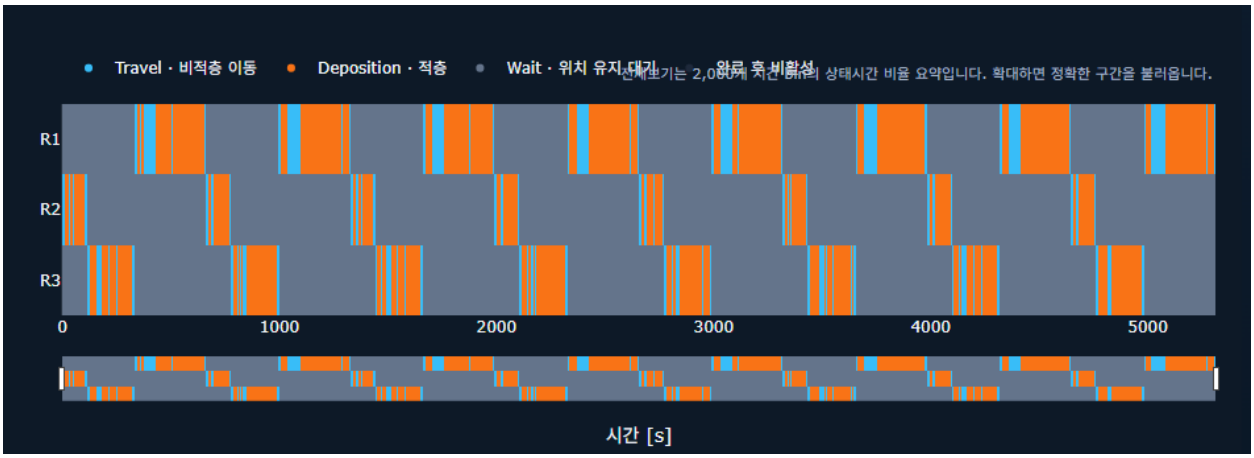
| **모델명** | **Makespan (DD:HH:MM:SS)** | **계산 시간 (DD:HH:MM:SS)** | **Validator 적용 결과 (Pass or Fail)** |
|---|---|---|---|
| 01 (Honeycomb) | 00:00:42:57 | 00:00:00:07 | PASS |
| 02 (Spiral) | 00:01:28:31 | 00:00:00:12 | PASS |
| 03 (Triangular truss) | 00:01:07:50 | 00:00:00:05 | PASS |
| 04 (Motor inner) | 00:20:06:15 | 00:00:01:07 | PASS |
| 05 (Twisted shell) | 05:01:23:21 | 00:00:08:35 | PASS |
| 06 (Angle frame) | 00:02:40:19 | 00:00:00:23 | PASS |

  - Validator 적용 결과 캡쳐 화면

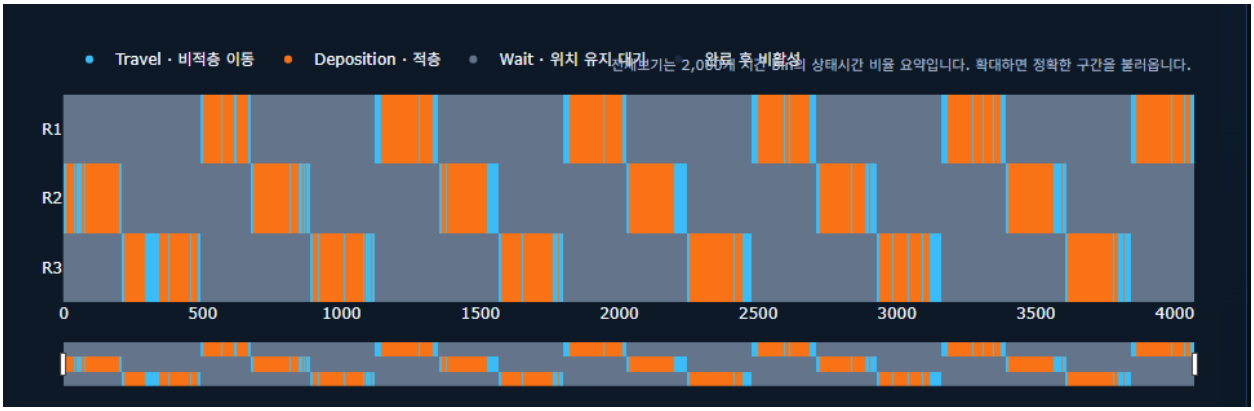
◆ 01



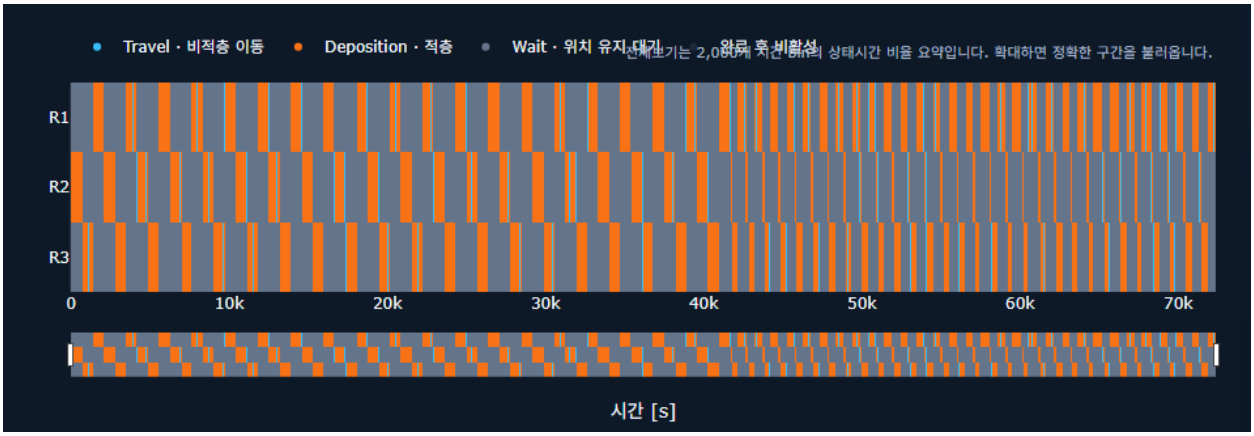

02


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
R1
R2
R3
0
1000
2000
3000
4000
5000
시간 [s]

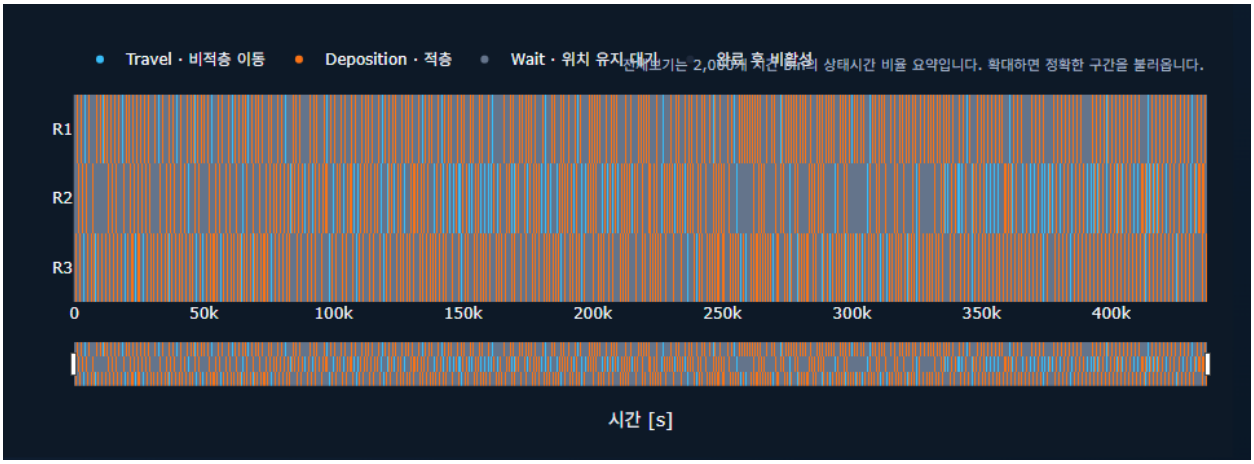
03


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
R1
R2
R3
0
500
1000
1500
2000
2500
3000
3500
4000

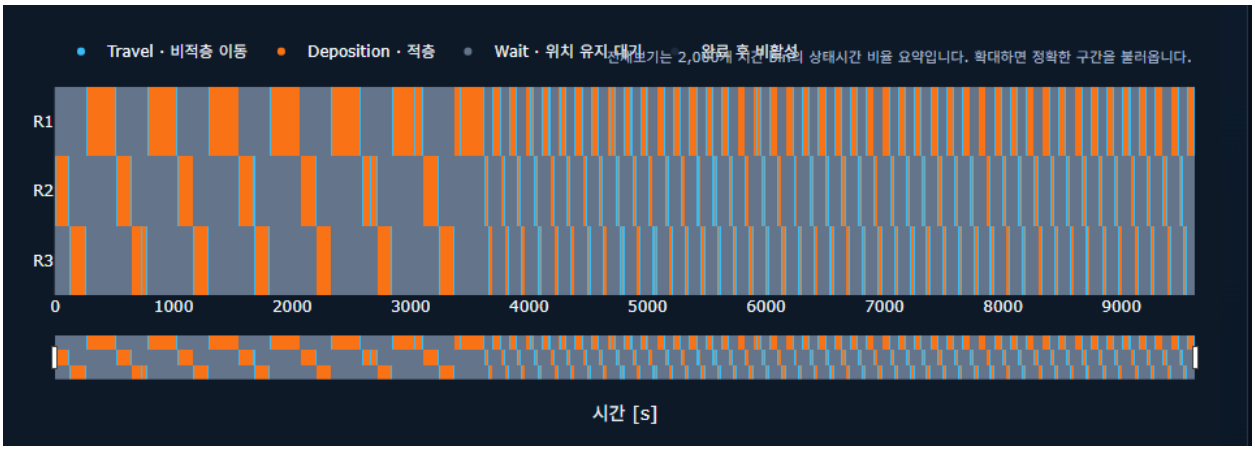
◆ 04


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
R1
R2
R3
0
10k
20k
30k
40k
50k
60k
70k
시간 [s]

◆ 05


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 처리 후의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
50k
100k
150k
200k
250k
300k
350k
400k
시간 [s]

● 06


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 처리 후의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
1000
2000
3000
4000
5000
6000
7000
8000
9000
시간 [s]

수도코드

# Main 1.0.2 - training and environment

Paper-style algorithm | STL preprocessing, layer observations, PPO actions, idle-time reward, and final Validator

## Algorithm 1. Main 1.0.2 - preprocessing and PPO training

```
Input: training STL/config set D_tr; validation set D_val; rollout length n_steps
For each scenario q in D_tr and D_val, parse config.yaml and validate robot/process parameters
Load target.stl with Trimesh; concatenate scene meshes; merge vertices and duplicate faces
Remove degenerate faces and unused vertices; repair normals; optionally fill holes
Check finite coordinates, watertight policy, build-plane bounds, and mesh validity
Choose evaluation layer indices; slice at build_plane + (k + 0.5) * layer_height
Normalize each section polygon using snap tolerance and area epsilon; discard empty layers
Cache a 16 x 16 fractional-area raster for each target layer
Initialize 3 robots at home; initialize PPO policy pi_theta, MultiDiscrete([8,3,4,2,2,3,6])
For PPO update u = 1, ..., U do
    Reset a training STL episode; initialize schedule and cumulative geometry state
    For each layer k do
        Observe progress, makespan, IoU, layer bounds/area/perimeter/raster and robot/process limits
        Sample action: angle, spacing, corner, reversal, partition axis/quota, robot order
        Generate inset contours and clipped hatch; split long segments; order from chosen corner
        Assign segments to reachable robots; rebalance unreachable requested assignments
        Build travel/deposition/wait routes; validate reach, trajectory rules and collisions
        On collision retry the same layer with serialized starts; reject if still infeasible
        Compute this layer deposited geometry, layer IoU and duration
        Compute robot activity utilization u_r and idle_factor = 1 + sum(idle_r)/duration
        r_layer = IoU - 0.1 * duration * idle_factor / reference_time
        On final layer run full WAAM Validator; add +1 for PASS or -1 for FAIL/ERROR
        Append layer routes with time offset; update observation and PPO rollout buffer
    end for; update theta with PPO; periodically score validation checkpoints by pass rate
end for; export best policy, trajectories, validation JSON and Gantt charts
```

## Procedure 1. Environment preprocessing and one-layer transition

```
ENV_INIT(q): parse config; clean and validate STL mesh; determine and slice requested layers
    Store normalized polygons, layer heights, robot bases/homes, reach and collision limits
    Build observation schema; lazily cache each layer 16 x 16 raster
LAYER_STEP(state, action): decode 7 action fields and select current target layer
    Generate path; assign segments; construct synchronized T/D/W routes for 3 robots
    Check local reach and collision; retry serialized schedule or reject with invalid reward -2
    Compute local deposited geometry and shape metrics; update cumulative geometry and IoU
    Compute reward = IoU - 0.1 * duration * (1 + sum(idle)/duration) / reference_time
    If final layer, export STL/config/trajectory bundle and call official WAAM Validator
    Add terminal reward (+1 PASS, -1 FAIL/ERROR); return next observation, reward and done
```

## Notation and implementation notes

- Preprocessing follows load_target_mesh, determine_evaluation_layers, slice_target_layers, and normalize_polygon.
- Observation includes layer geometry and a cached raster plus schedule, robot, process, and validation features.
- One MultiDiscrete action (7 fields) generates one complete layer path.
- Idle factor is 3 when one robot is active for the full layer, it is 1 when all three are active.
- Local checks reject infeasible candidates; the official whole-trajectory Validator is final authority.

- ㄴ이 알고리즘을 이 문제에 적용한 이유
  - ◆
- 알고리즘에 대한 세부 설명(특징, 장단점 등)
  - ◆

■ 기타

■ Parameters

| 파라미터 | 현재 설정 | 의미 |
|---|---|---|
| 학습 업데이트 | - / -회 | 요청 횟수 / 완료 횟수 |
| 학습 타임스텝 | 1024 / 1024 | 완료 / 요청 |
| n_steps | 64 | PPO rollout 단계 수 |
| batch_size | 32 | 미니배치 크기 |
| learning_rate | 0.0003 | 학습률 |
| gamma | 0.99 | 할인율 |
| gae_lambda | 0.95 | GAE 람다 |
| seed | 42 | 재현용 시드 |
| device | cuda | 정책 네트워크 계산 장치 |
| early_stopping | - | 조기 종료 설정 |
| eval_every_updates | 4 | 평가 주기 |
| eval_episodes | 1 | 평가 STL 별 반복 횟수 |
| geometry_device | cpu | 기하·Validator 계산 장치 |
| training STL | 6 | 학습 입력 STL 개수 |
| validation STL | 0 | 학습 중 평가 STL 개수 |
| placement_enabled | False | 전역 XY placement PPO 사용 여부 |
| reward_formula | - | 학습 보상식 ID |
| best_step | 256 | 최고 검증 점수 체크포인트 단계 |
| best_model | best_model.zip | 실행 시 불러오는 모델 |
| training_status | completed | 학습 기록 상태 |

■

■

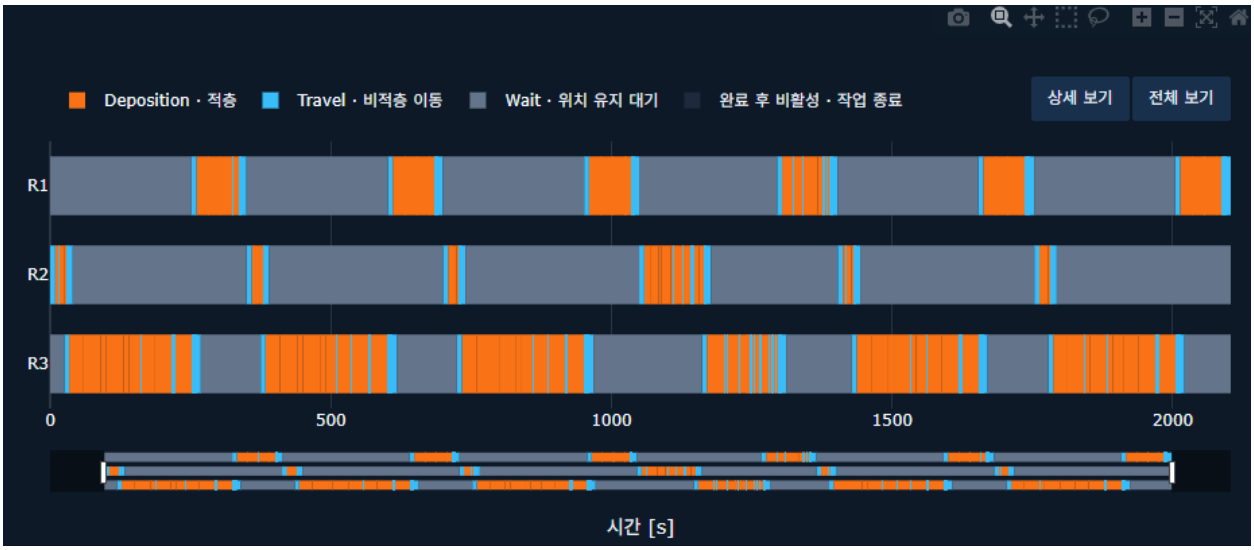
- **알고리즘 2 번**
  - ■

- 알고리즘 1 번 | Minor1 Placement and Scheduler PPO
  - 모델별 적용 결과

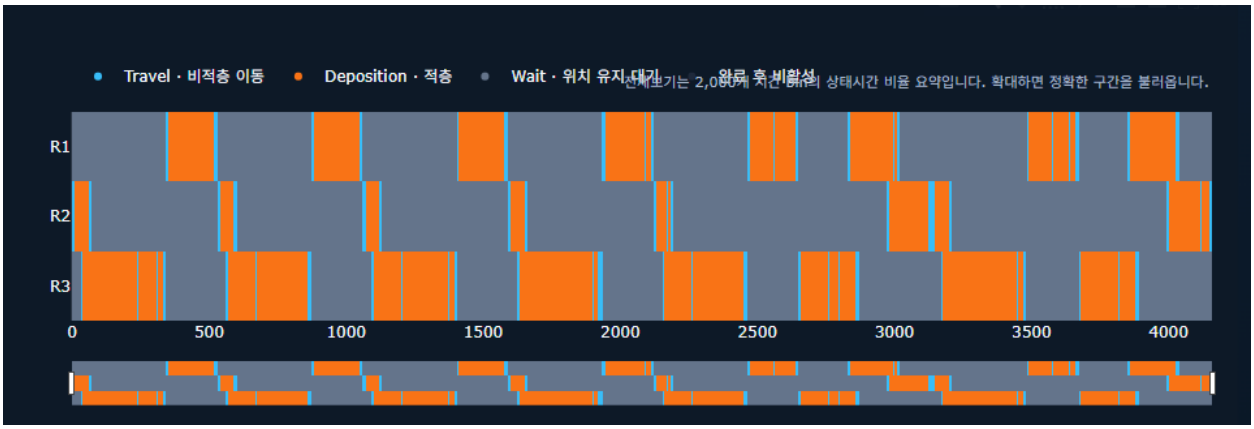
| 모델명 | Makespan (DD:HH:MM:SS) | 계산 시간 (DD:HH:MM:SS) | Validator 적용 결과 (Pass or Fail) |
|---|---|---|---|
| 01 (Honeycomb) | 00:00:35:03 | 00:00:00:13 | PASS |
| 02 (Spiral) | 00:01:09:18 | 00:00:00:25 | PASS |
| 03 (Triangular truss) | 00:00:57:30 | 00:00:00:14 | PASS |
| 04 (Motor inner) | 00:15:02:28 | 00:00:02:59 | PASS |
| 05 (Twisted shell) | 02:03:13:53 | 00:00:37:06 | PASS |
| 06 (Angle frame) | 00:02:12:27 | 00:00:01:26 | PASS |

  - Validator 적용 결과 캡쳐 화면

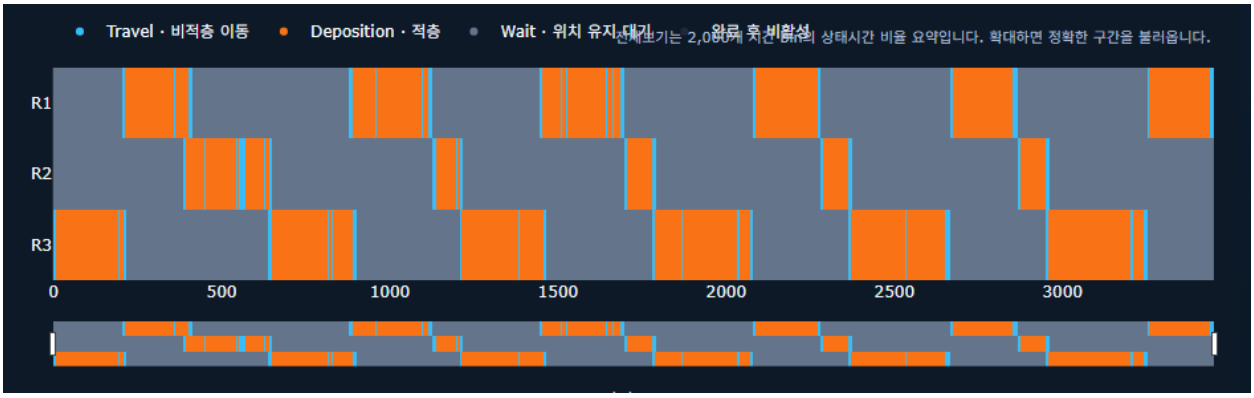
◆ 01



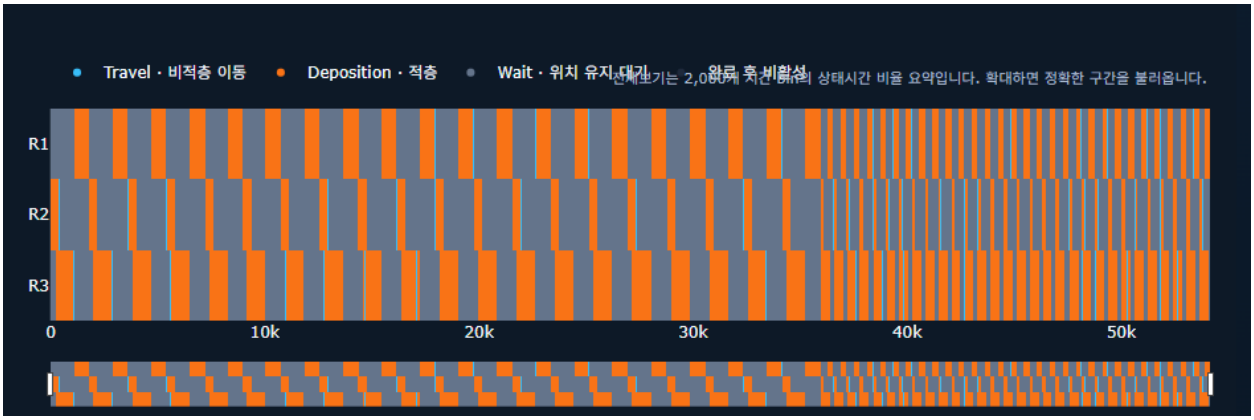

◆ 02



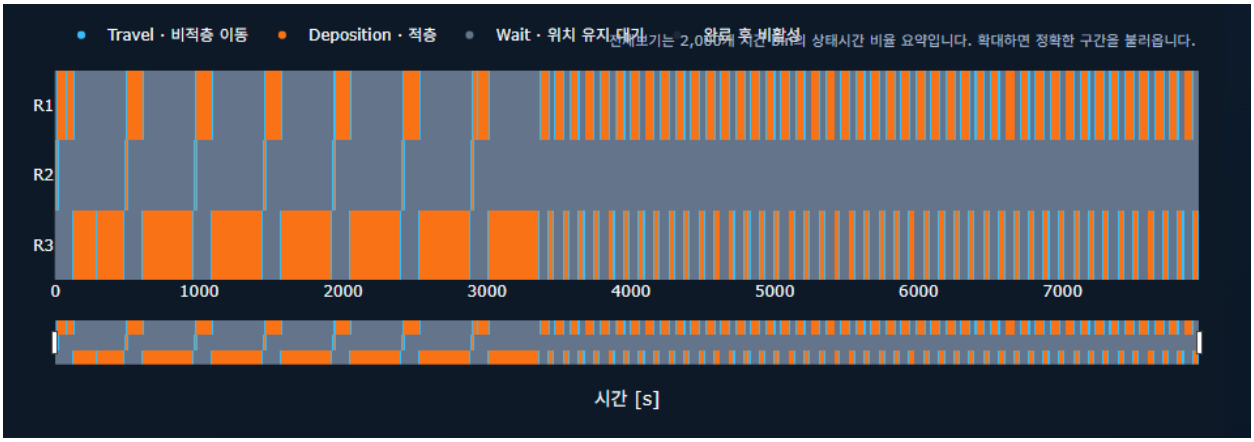
◆ 03


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
완료 후 비활성
선택 구간의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
500
1000
1500
2000
2500
3000

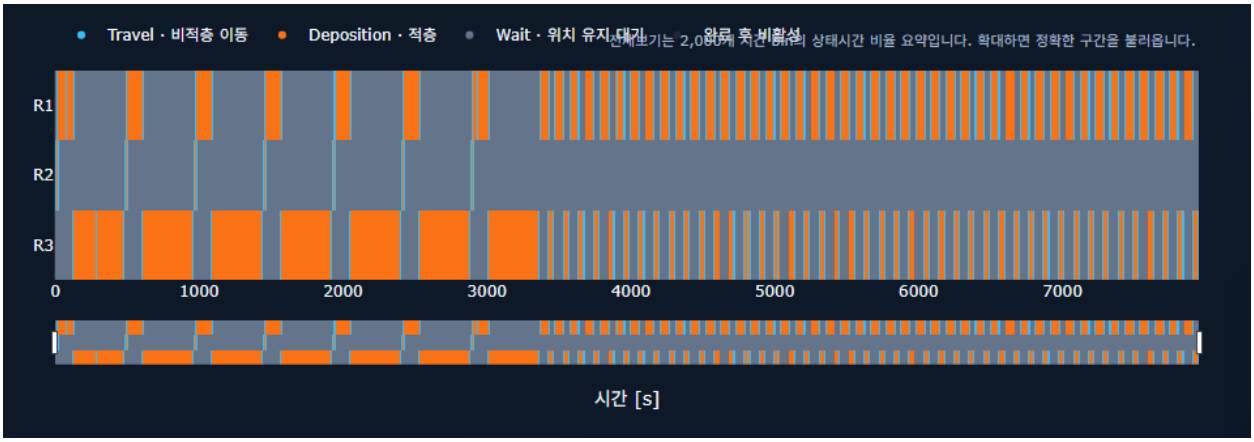
◆ 04


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
완료 후 비활성
선택 구간의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
10k
20k
30k
40k
50k

◆ 05


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
완료 후 비활성
선택 구간의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
1000
2000
3000
4000
5000
6000
7000
시간 [s]

◆ 06


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
R1
R2
R3
0
1000
2000
3000
4000
5000
6000
7000
시간 [s]

● 수도코드

## Algorithm 1. Minor 1 — joint path and placement PPO

```
Input: training STLs D_tr, validation set D_val, updates K
Initialize path policy πθ from compatible 1.0.2 weights
Initialize placement policy πϕ; placement action is global (Δx, Δy)
for update k = 1, …, K do
    Freeze πϕ; collect 64 layer steps using πθ and fixed placement
    Update θ with PPO using the layer rewards R_layer
    Freeze πθ
    for b = 1, …, 2 STL episodes do
        Select one STL q and predict Δxy ← πϕ(observe(q))
        Translate all of q by Δxy before slicing; preserve Z and shape
        Execute πθ for every layer; collect {R_layer} for STL q
        Set R_place ← mean({R_layer} from this STL episode only)
    end for; update ϕ with PPO using R_place
    if k mod 4 = 0 then evaluate (πθ, πϕ) on D_val
        Select best pair: pass rate, normalized makespan, overlap
    end if
end for
return best path-policy / placement-policy pair
```

- ㄴ이 알고리즘을 이 문제에 적용한 이유
  - ◆
- 알고리즘에 대한 세부 설명(특징, 장단점 등)
  - ◆

■ 기타

■ Parameters

| 파라미터 | 현재 설정 | 의미 |
|---|---|---|
| 학습 업데이트 | 28 / 40 회 | 요청 횟수 / 완료 횟수 |
| 학습 타임스텝 | 1792 / 2560 | 완료 / 요청 |
| n_steps | 64 | PPO rollout 단계 수 |
| batch_size | 32 | 미니배치 크기 |
| learning_rate | 0.0003 | 학습률 |
| gamma | 0.99 | 할인율 |
| gae_lambda | 0.95 | GAE 람다 |
| seed | 42 | 재현용 시드 |
| device | cuda | 정책 네트워크 계산 장치 |
| early_stopping | False | 조기 종료 설정 |
| eval_every_updates | 4 | 평가 주기 |
| eval_episodes | 1 | 평가 STL 별 반복 횟수 |
| geometry_device | cpu | 기하·Validator 계산 장치 |
| training STL | 6 | 학습 입력 STL 개수 |
| validation STL | 6 | 학습 중 평가 STL 개수 |

| | | |
|---|---|---|
| placement_enabled | True | 전역 XY placement PPO 사용 |
| placement_steps_per_update | 2 | 업데이트당 placement episode 수 |
| placement_every | 1 | placement 학습 주기 |
| placement PPO n_steps | 2 | placement rollout 길이 |
| placement PPO n_epochs | 4 | placement PPO 반복 횟수 |
| reward_formula | iou_gt_0.9_utilization_over_makespan_v1 | layer 보상식 ID |
| placement_reward | mean_of_layer_rewards_from_this_stl_only | 동일 STL 의 layer reward 평균 |
| best_step | 1024 | 최고 검증 점수 체크포인트 단계 |
| best_model | best_model.zip | 실행 시 불러오는 모델 |
| training_status | started | 학습 기록 상태 |



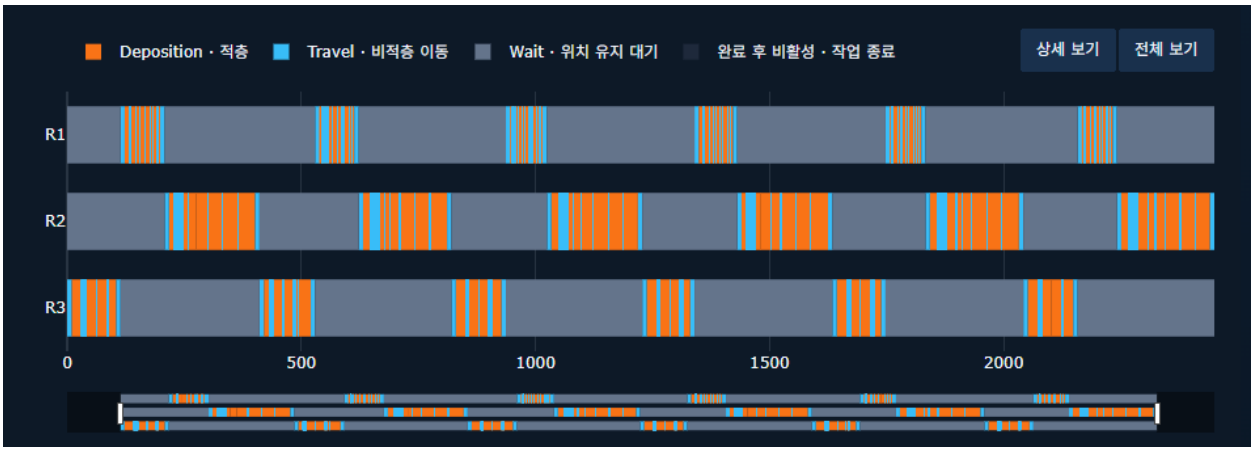
- **알고리즘 2 번**
  - ■

- 알고리즘 1 번 | Minor2 Reward Only PPO
  - 모델별 적용 결과

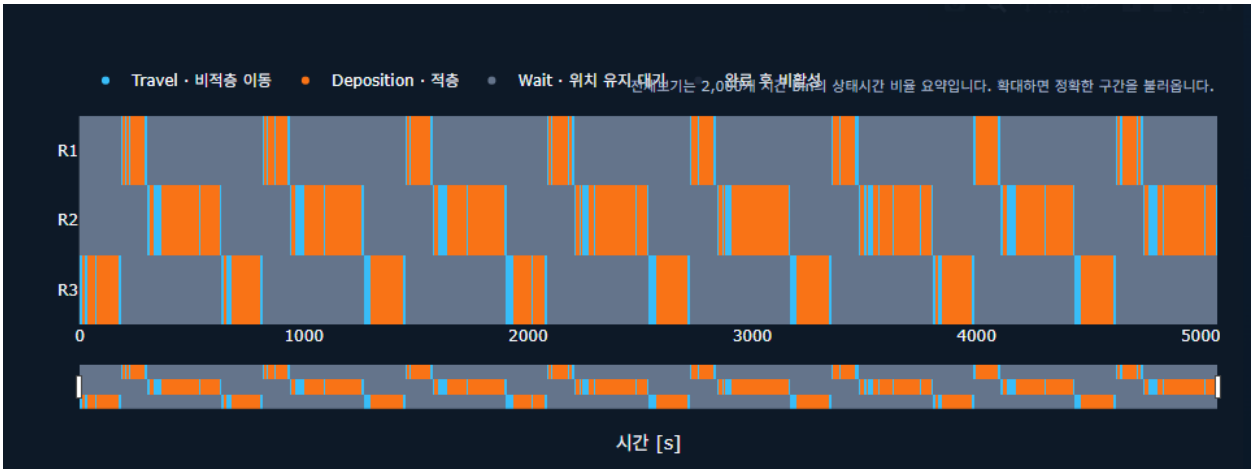
| 모델명 | Makespan (DD:HH:MM:SS) | 계산 시간 (DD:HH:MM:SS) | Validator 적용 결과 (Pass or Fail) |
|---|---|---|---|
| 01 (Honeycomb) | 00:00:40:50 | 00:00:00:05 | PASS |
| 02 (Spiral) | 00:01:24:34 | 00:00:00:10 | PASS |
| 03 (Triangular truss) | 00:01:06:16 | 00:00:00:06 | PASS |
| 04 (Motor inner) | 00:19:16:04 | 00:00:01:25 | PASS |
| 05 (Twisted shell) | 04:12:27:22 | 00:00:07:14 | PASS |
| 06 (Angle frame) | 00:02:41:07 | 00:00:00:16 | PASS |

  - Validator 적용 결과 캡쳐 화면

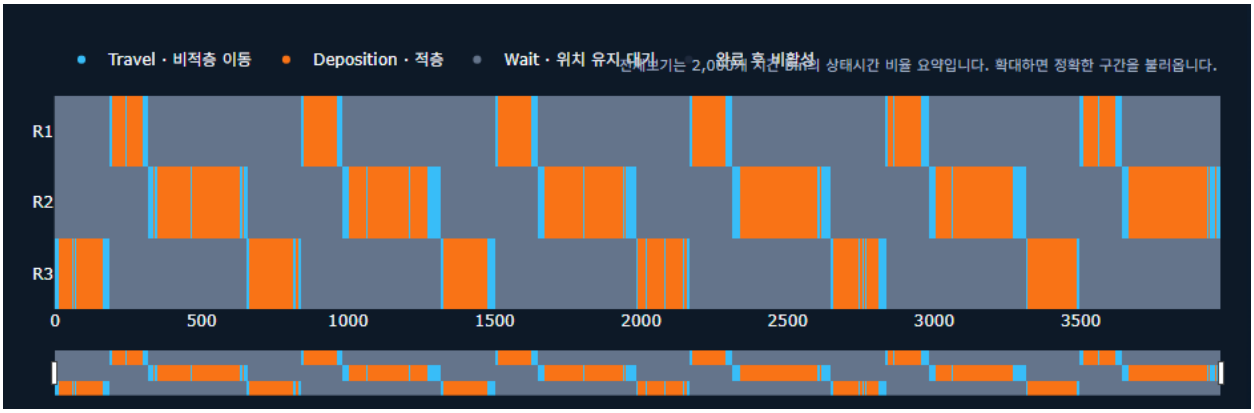
◆ 01



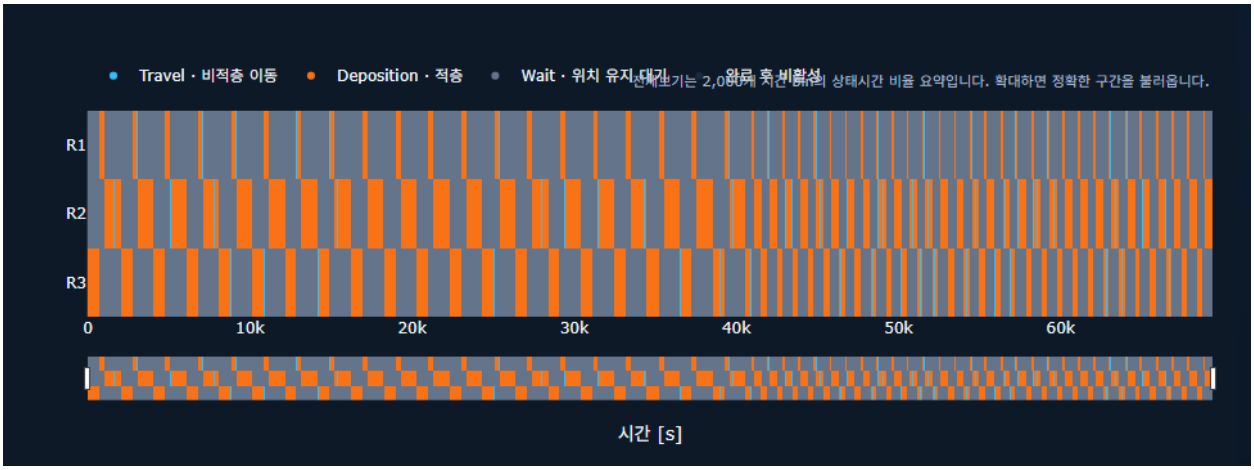

◆ 02



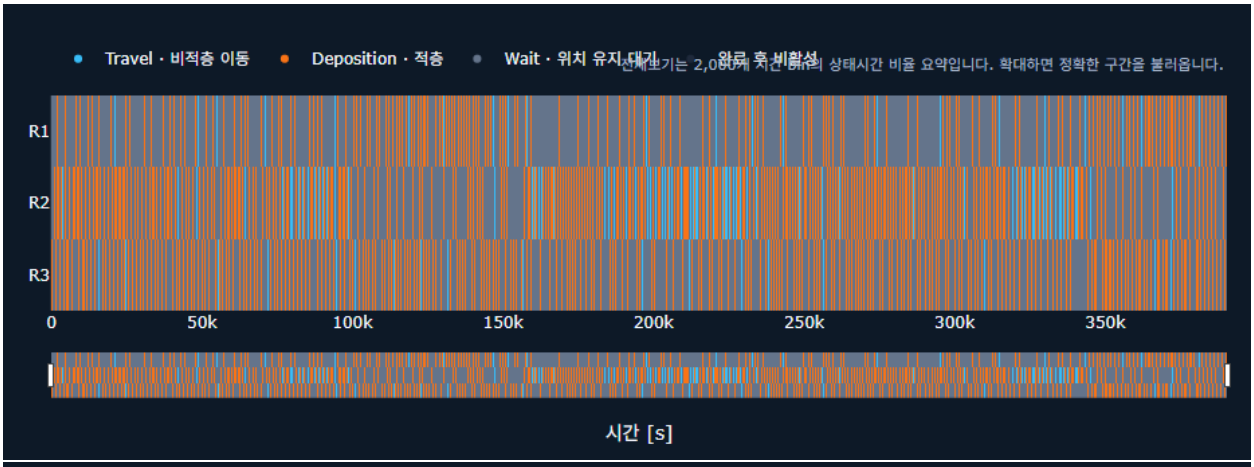
◆ 03

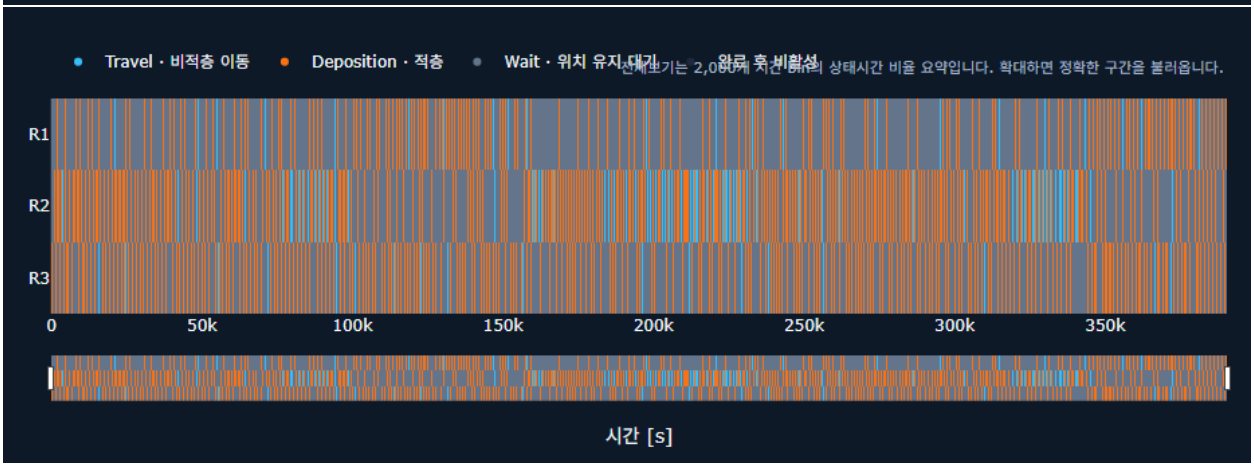

Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 처리 bin의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
500
1000
1500
2000
2500
3000
3500

◆ 04

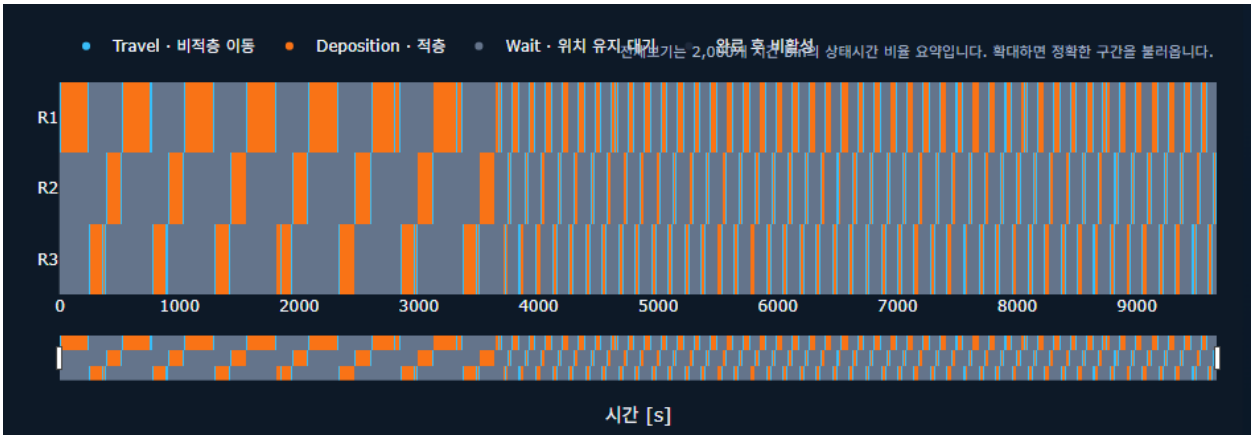

Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 처리 bin의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
10k
20k
30k
40k
50k
60k
시간 [s]

◆ 05


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 지점 sm의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
50k
100k
150k
200k
250k
300k
350k
시간 [s]


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 지점 sm의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
50k
100k
150k
200k
250k
300k
350k
시간 [s]

◆ 06


Travel · 비적층 이동
Deposition · 적층
Wait · 위치 유지 대기
전체보기는 2,000개 지점 sm의 상태시간 비율 요약입니다. 확대하면 정확한 구간을 불러옵니다.
R1
R2
R3
0
1000
2000
3000
4000
5000
6000
7000
8000
9000
시간 [s]

● 수도코드

■

# Minor2 - reward-only layer PPO

Paper-style algorithm | three-robot path generation and IoU-gated time/utilization reward

**Algorithm 2. Minor2 - reward-only layer PPO**

```
Input: training STL/config set D_tr; held-out validation set D_val; initial
      Main PPO weights
Initialize layer-path policy pi_theta; retain original action and
      route-generation environment
For PPO update k = 1, ..., K do
  Collect one rollout of n_steps layer transitions from training STL
      episodes
  For each layer, observe geometry/raster, schedule progress, robot reach
      and process settings
    Select hatch angle, spacing, start corner, reversal, partition axis,
      quota and robot order
    Generate contours/hatch and partition segments among reachable robots
    Build concurrent routes; check reach/collision; retry serialized starts
      on collision
    Reject infeasible paths with invalid-action penalty -2
    Otherwise compute deposited geometry, layer IoU I, duration T and
      utilization U_D
    If I > 0.9, reward = 10 * U_D * reference_time / T
    Else terminal Validator PASS = +1; FAIL/ERROR = -1; no result = 0
    Append layer schedule; return next observation, reward and completion
      flag
  End for; update pi_theta with PPO
  Every four updates evaluate held-out STLs; select checkpoint by Validator
      pass rate
End for; return best path policy and export training-STL trajectories
```

- ㄴ이 알고리즘을 이 문제에 적용한 이유
  - ◆
- 알고리즘에 대한 세부 설명(특징, 장단점 등)
  - ◆

■ 기타

■ Parameters

| 파라미터 | 현재 설정 | 의미 |
|---|---|---|
| 학습 업데이트 | 40 / 40 회 | 요청 횟수 / 완료 횟수 |
| 학습 타임스텝 | 2560 / 2560 | 완료 / 요청 |
| n_steps | 64 | PPO rollout 단계 수 |
| batch_size | 32 | 미니배치 크기 |
| learning_rate | 0.0003 | 학습률 |
| gamma | 0.99 | 할인율 |
| gae_lambda | 0.95 | GAE 람다 |
| seed | 42 | 재현용 시드 |
| device | cuda | 정책 네트워크 계산 장치 |
| early_stopping | - | 조기 종료 설정 |
| eval_every_updates | 4 | 평가 주기 |
| eval_episodes | 1 | 평가 STL 별 반복 횟수 |
| geometry_device | cpu | 기하·Validator 계산 장치 |
| training STL | 6 | 학습 입력 STL 개수 |
| validation STL | 6 | 학습 중 평가 STL 개수 |

| | | |
|---|---|---|
| placement_enabled | False | 전역 XY placement PPO 사용 여부 |
| reward_formula | iou_gt_0.9_deposition_utilization_over_layer_makespan_v1 | 학습 보상식 ID |
| best_step | 256 | 최고 검증 점수 체크포인트 단계 |
| best_model | best_model.zip | 실행 시 불러오는 모델 |
| training_status | completed | 학습 기록 상태 |



- **알고리즘 2 번**
  - ■

**Trajectory files appendix**

Trajectory CSV files are stored by experiment and input STL. The experiment folders retain their original evaluation trees; separate 1.0.2 exports are listed below.

**Main 1.0.2**

main/stl_files/job_00-job_05/episode_000/input/trajectory.csv; main/validation_data/job_00-job_05/episode_000/input/trajectory.csv

**Minor1**

minor1/stl_files/job_00-job_05/episode_000/input/trajectory.csv; minor1/validation_data/job_00-job_05/episode_000/input/trajectory.csv

**Minor2**

minor2/stl_files/job_00-job_05/episode_000/input/trajectory.csv; minor2/validation_data/job_00-job_05/episode_000/input/trajectory.csv

**Main standalone exports**

main/stl_files/exported_trajectory/01-06/trajectory.csv

# Main 1.0.2 - preprocessing and PPO training

Paper-style algorithm | STL preprocessing, layer actions, idle-time reward and final Validator

## Algorithm 1. Main 1.0.2 - preprocessing and PPO training

```
Input: training STL/config set D_tr; held-out validation set D_val; PPO
      rollout settings
For each job, parse config, load/repair STL mesh, validate geometry, process
      and robot limits
Select evaluation layers; slice each section at build_plane + (k + 0.5) *
      layer_height
Normalize layer polygons, discard empty layers, and cache a 16 x 16
      fractional-area raster
Initialize three robots at home and PPO policy pi_theta with
      MultiDiscrete([8,3,4,2,2,3,6])
For PPO update u = 1, ..., U do
  Reset a training STL episode; initialize schedule and cumulative deposited
      geometry
  For each target layer k do
    Observe progress, makespan, IoU, layer geometry/raster, robot reach and
      process settings
    Select hatch angle, spacing, start corner, reversal, partition axis,
      quota and robot order
    Generate inset contours and clipped hatch; split long segments and
      assign reachable robots
    Build travel/deposition/wait routes; check reach, trajectory rules and
      collisions
    On collision retry serialized starts; reject infeasible routes with
      reward -2
    Compute deposited geometry, layer IoU, duration and robot utilization
    Set idle_factor = 1 + sum(robot idle seconds) / layer duration
    Reward = IoU - 0.1 * duration * idle_factor / reference_time
    On final layer run whole-schedule Validator; add +1 for PASS or -1 for
      FAIL/ERROR
    Append routes with time offsets; update observation and PPO rollout
      buffer
  End for; PPO-update policy and periodically score held-out validation
      checkpoints
End for; export best policy, trajectories, validation JSON and Gantt charts
```

### Notation and implementation notes

- Preprocessing loads, repairs, validates, slices and normalizes each target STL before training.
- One PPO step generates one complete layer path; three robots share the layer schedule.
- The official whole-trajectory WAAM Validator is the final authority.

# Minor1 - joint placement and layer PPO

Paper-style algorithm | shared STL translation, learned path policy and collision-aware schedule search

## Algorithm 1. Minor1 - joint placement and layer PPO

```
Input: disjoint training jobs D_tr; held-out validation set D_val; layer
      policy pi_l; placement policy pi_p
Initialize pi_l and pi_p (optionally load compatible weights); initialize
      layer and placement environments
For PPO update u = 1, ..., U do
  Collect n_steps layer transitions under current placement; PPO-update
      layer/path policy pi_l
  If u is a placement-update interval, collect n_place full-STL episodes
      (n_place >= 2)
  For each placement episode, select training STL q and observe
      mesh/workspace/robot features
    Sample shared (dx,dy) from pi_p; clip and scale by workspace radius
    Project translation so every mesh-hull vertex remains inside the
      workspace
    Translate the entire STL by (dx,dy,0) before slicing; all layers share
      the shift
    For each target layer k, pi_l selects path parameters and robot
      partition; generate segments
      Build original concurrent schedule; use serial fallback if collision
      requires it
      Try nearest-base/load-balanced assignment and reorder routes
      near-to-far
      Try concurrent starts and bounded staggered offsets in two robot
      priority orders
      Reject candidates failing reach, trajectory or collision checks;
      retain shortest feasible
      Keep all generated deposition segments; search changes
      assignment/order/timing only
      If IoU > 0.9, r = 10 * mean deposition utilization * reference time /
      layer duration
      Else terminal Validator PASS = +1; FAIL/ERROR = -1; unavailable = 0
    End for; placement reward = mean layer rewards from this STL only
  End for; PPO-update pi_p from full-STL placement rewards
  Evaluate held-out jobs; retain best paired checkpoint by validation key
End for; export full trajectories and run official WAAM Validator
```

### Notation and implementation notes

- Two PPO policies are learned: global XY placement and layer/path selection.
- One shared XY shift is applied to the complete STL before slicing.
- The collision-aware schedule candidate search is bounded heuristic search, not a learned scheduler.

# Minor2 - reward-only layer PPO

Paper-style algorithm | three-robot path generation and IoU-gated time/utilization reward

## Algorithm 2. Minor2 - reward-only layer PPO

```
Input: training STL/config set D_tr; held-out validation set D_val; initial
      Main PPO weights
Initialize layer-path policy pi_theta; retain original action and
      route-generation environment
For PPO update k = 1, ..., K do
  Collect one rollout of n_steps layer transitions from training STL
      episodes
  For each layer, observe geometry/raster, schedule progress, robot reach
      and process settings
    Select hatch angle, spacing, start corner, reversal, partition axis,
      quota and robot order
    Generate contours/hatch and partition segments among reachable robots
    Build concurrent routes; check reach/collision; retry serialized starts
      on collision
    Reject infeasible paths with invalid-action penalty -2
    Otherwise compute deposited geometry, layer IoU I, duration T and
      utilization U_D
    If I > 0.9, reward = 10 * U_D * reference_time / T
    Else terminal Validator PASS = +1; FAIL/ERROR = -1; no result = 0
    Append layer schedule; return next observation, reward and completion
      flag
  End for; update pi_theta with PPO
  Every four updates evaluate held-out STLs; select checkpoint by Validator
      pass rate
End for; return best path policy and export training-STL trajectories
```

## Notation and implementation notes

- Minor2 changes layer rewards and has no global STL-placement policy.
- The performance reward applies only when layer IoU is strictly greater than 0.9.
- Mean deposition utilization includes deposition time only; travel and waiting are excluded.
- The whole-STL Validator confirms final schedule validity.

# Minor1 과 1.1.1.e retry1 결과 비교

동일한 원본 학습 STL 6 개를 Minor1 최우수 체크포인트(1,024 step)와 1.1.1.e retry1 최종 체크포인트(2,560 step)로 평가한 결과이다. 두 모델의 정책, STL 배치 및 학습량이 다르므로 표의 차이는 방법 전체의 결과 차이다.

## 학습 STL Makespan

| **STL** | **Minor1**<br>**DD:HH:MM:SS** | **Retry1**<br>**DD:HH:MM:SS** | **변화율** | **Validator**<br>**Minor1 / Retry1** |
|---|---|---|---|---|
| 01 Honeycomb | 00:00:35:03 | 00:00:44:08 | +25.90% | PASS / PASS |
| 02 Spiral | 00:01:09:18 | 00:01:39:54 | +44.13% | PASS / PASS |
| 03 Triangular truss | 00:00:57:30 | 00:01:09:48 | +21.38% | PASS / PASS |
| 04 Motor inner | 00:15:02:29 | 00:17:52:37 | +18.85% | PASS / PASS |
| 05 Twisted shell | 02:03:13:53 | 01:14:20:19 | -25.17% | PASS / PASS |
| 06 Angle frame | 00:02:12:28 | 00:02:26:40 | +10.73% | PASS / PASS |

변화율 = (Retry1 Makespan / Minor1 Makespan − 1) × 100. 음수는 Retry1 의 제작시간 감소를 뜻한다.

## 사례별 계산 시간

| **STL** | **Minor1**<br>**DD:HH:MM:SS** | **Retry1**<br>**DD:HH:MM:SS** |
|---|---|---|
| 01 | 00:00:00:13 | 00:00:00:10 |
| 02 | 00:00:00:25 | 00:00:00:43 |
| 03 | 00:00:00:14 | 00:00:00:17 |
| 04 | 00:00:03:00 | 00:00:04:05 |
| 05 | 00:00:37:07 | 00:00:04:56 |
| 06 | 00:00:01:27 | 00:00:00:10 |

기록된 계산시간의 측정 구간은 버전별 실행기에 따라 다르다. Retry1 의 값은 trajectory 생성 구간이며 WAAM Validator 실행시간을 제외한다. 정확한 초 단위 값은 minor1_retry1_comparison.csv 에 있다.

공통 validation 6 개 Makespan

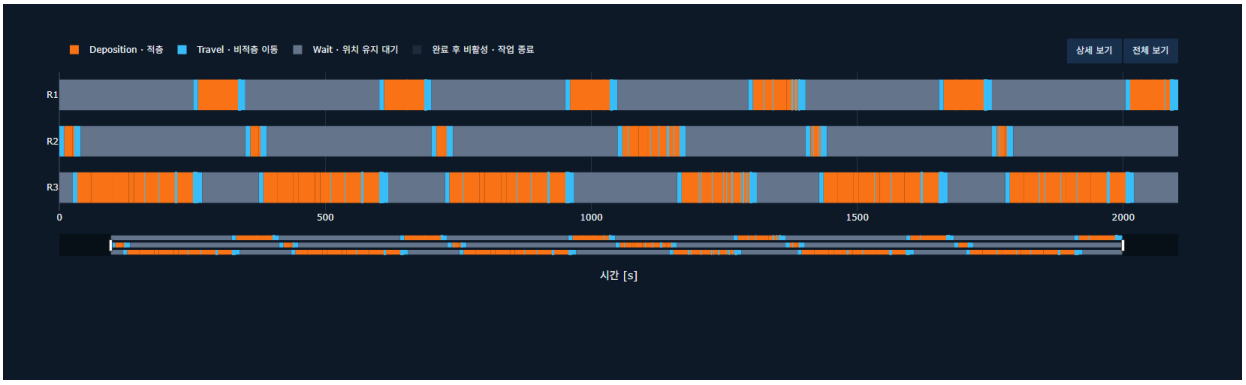
| 입력 | Minor1<br>DD:HH:MM:SS | Retry1<br>DD:HH:MM:SS | 변화율 | Validator |
|---|---|---|---|---|
| 00_sample | 00:00:00:43 | 00:00:01:02 | +43.95% | PASS / PASS |
| 01_rectangle | 00:00:08:20 | 00:00:09:13 | +10.42% | PASS / PASS |
| 02_triangle | 00:00:03:02 | 00:00:03:21 | +10.44% | PASS / PASS |
| 03_l_shape | 00:00:08:03 | 00:00:08:54 | +10.73% | PASS / PASS |
| 04_cross | 00:00:04:17 | 00:00:04:29 | +4.79% | PASS / PASS |
| 05_ring | 00:00:05:10 | 00:00:05:57 | +15.11% | PASS / PASS |

작성자 설명

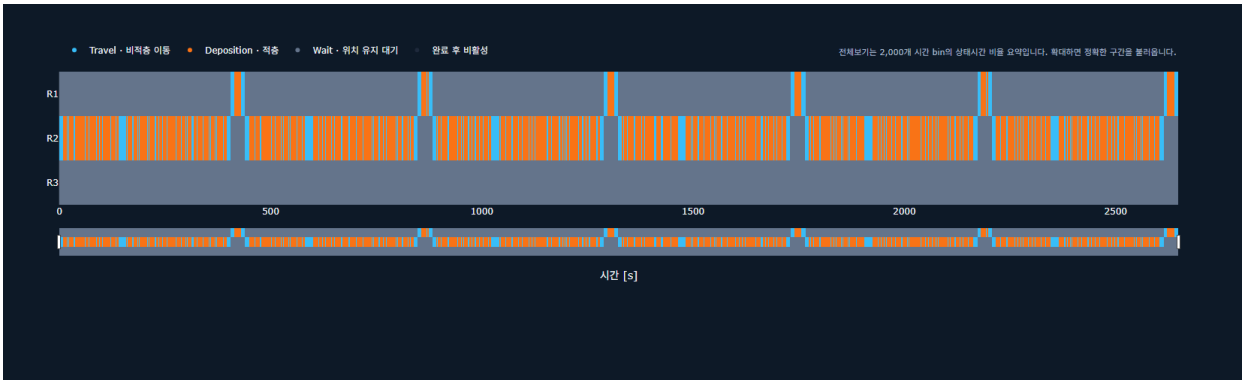
# Minor1 과 Retry1 WAAM Validator 간트 비교

01 Honeycomb | Minor1 00:00:35:03 | Retry1 00:00:44:08

Minor1



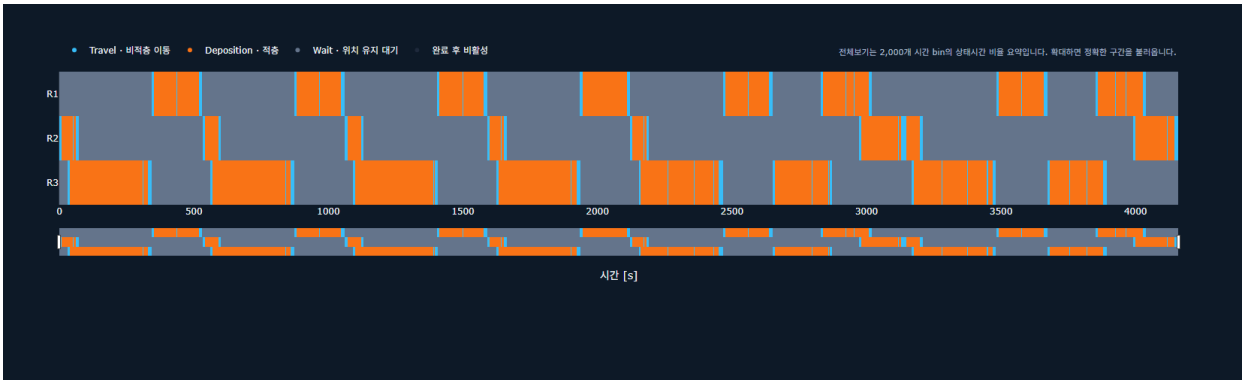

1.1.1.e retry1



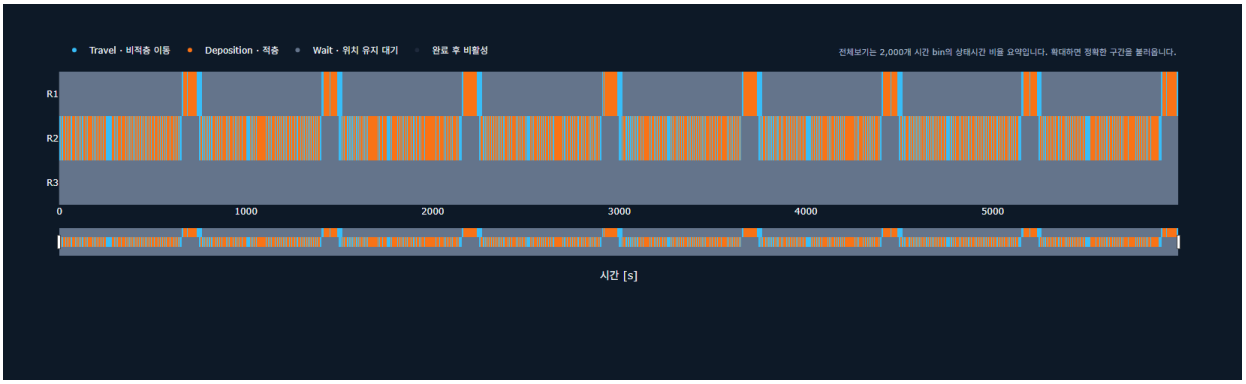
# Minor1 과 Retry1 WAAM Validator 간트 비교

02 Spiral | Minor1 00:01:09:18 | Retry1 00:01:39:54

Minor1



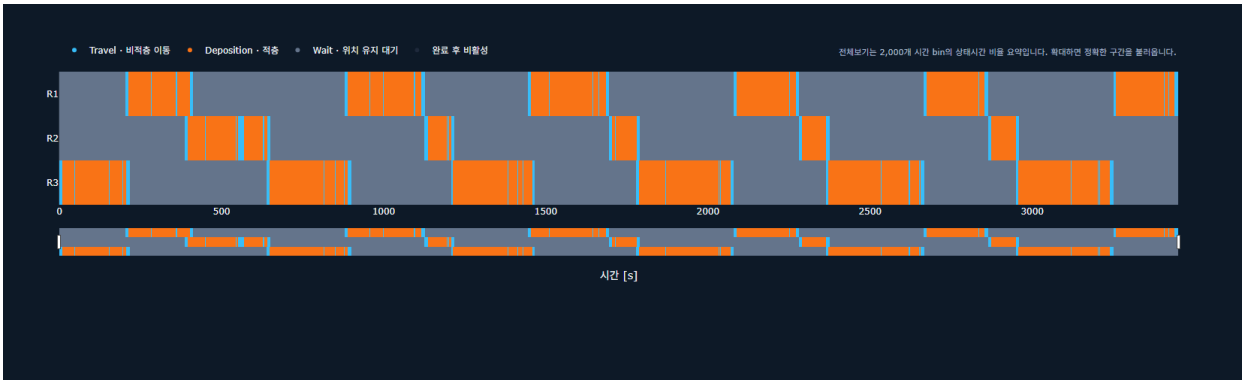

1.1.1.e retry1



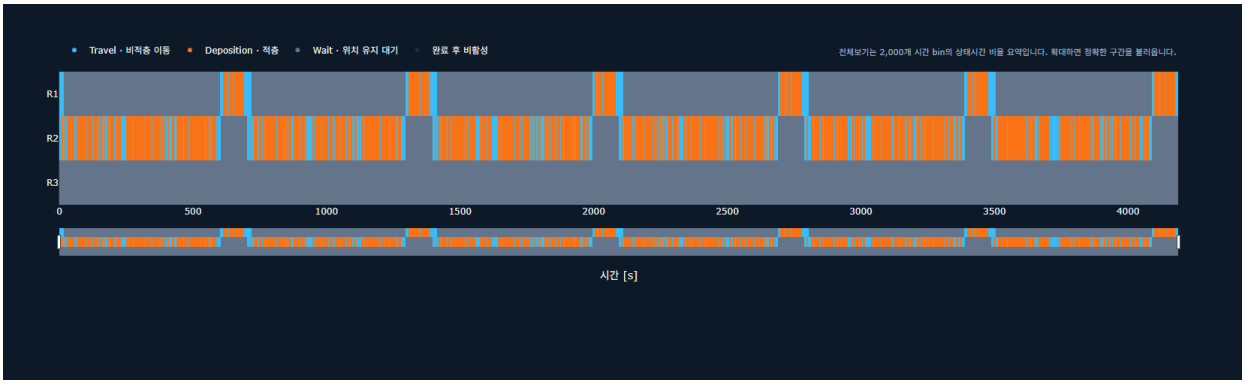
# Minor1 과 Retry1 WAAM Validator 간트 비교

03 Triangular truss | Minor1 00:00:57:30 | Retry1 00:01:09:48

Minor1



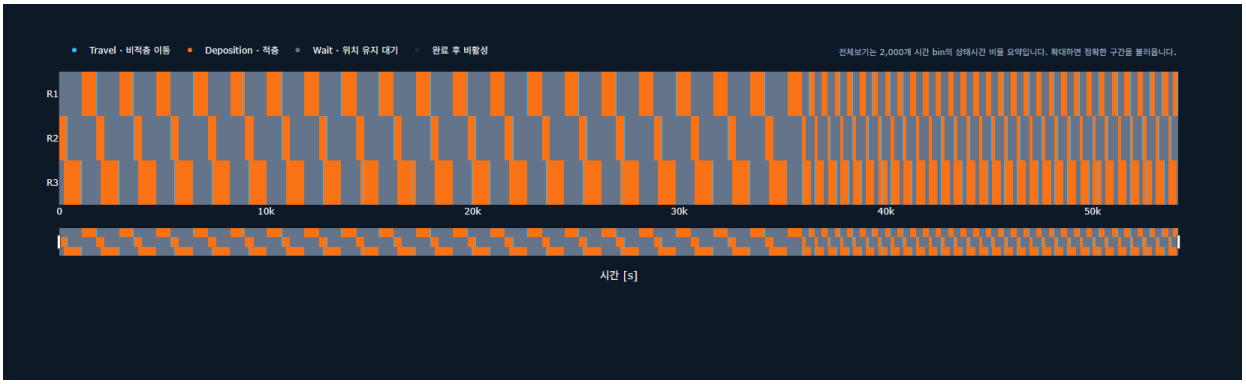

1.1.1.e retry1



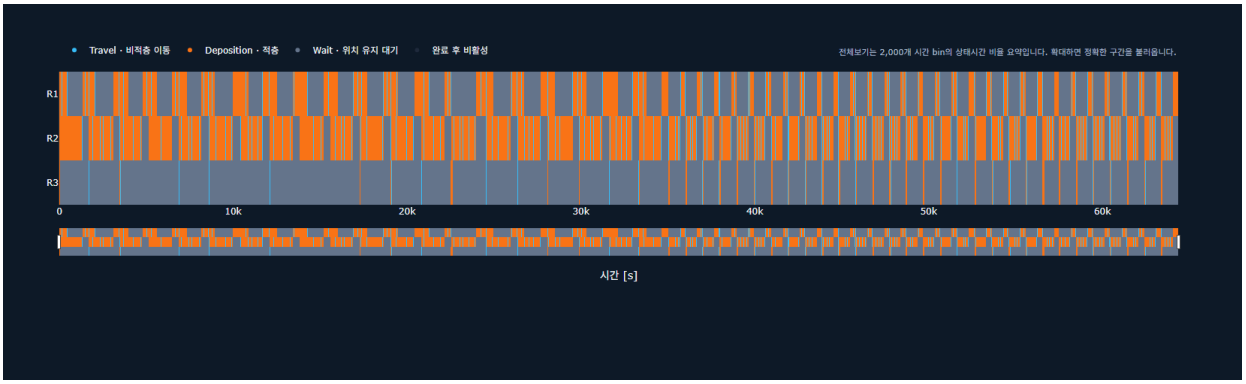
# Minor1 과 Retry1 WAAM Validator 간트 비교

04 Motor inner | Minor1 00:15:02:29 | Retry1 00:17:52:37

Minor1



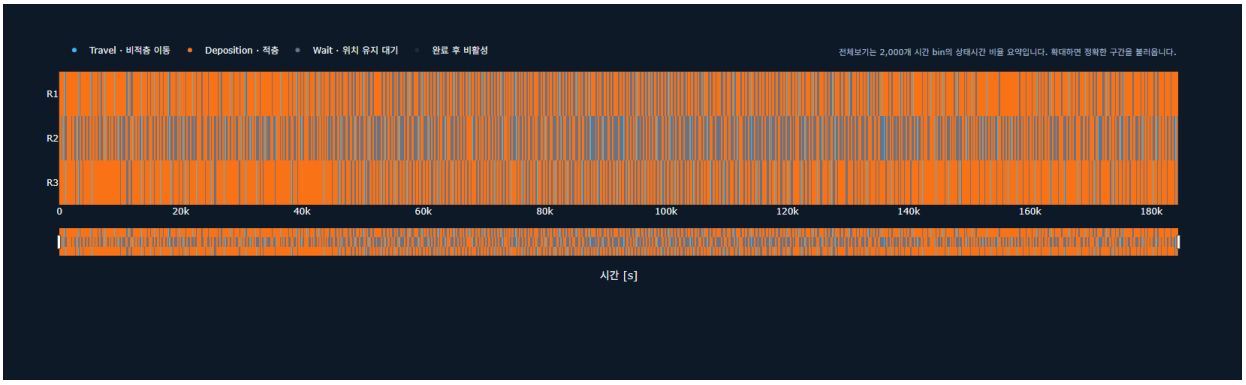

1.1.1.e retry1



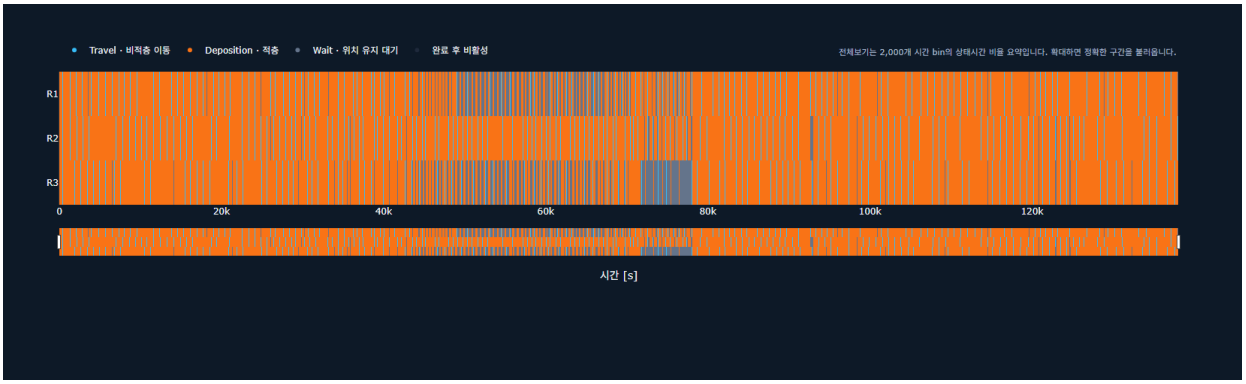
# Minor1 과 Retry1 WAAM Validator 간트 비교

05 Twisted shell | Minor1 02:03:13:53 | Retry1 01:14:20:19

Minor1



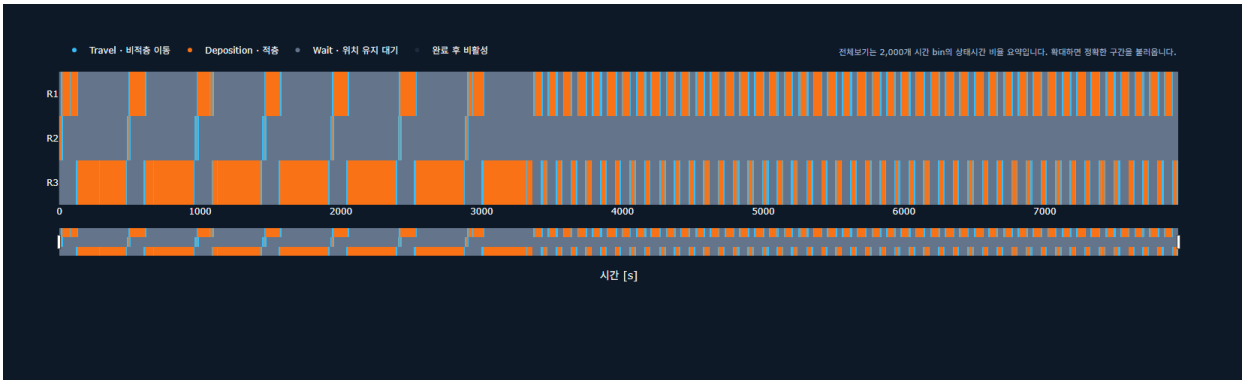

1.1.1.e retry1



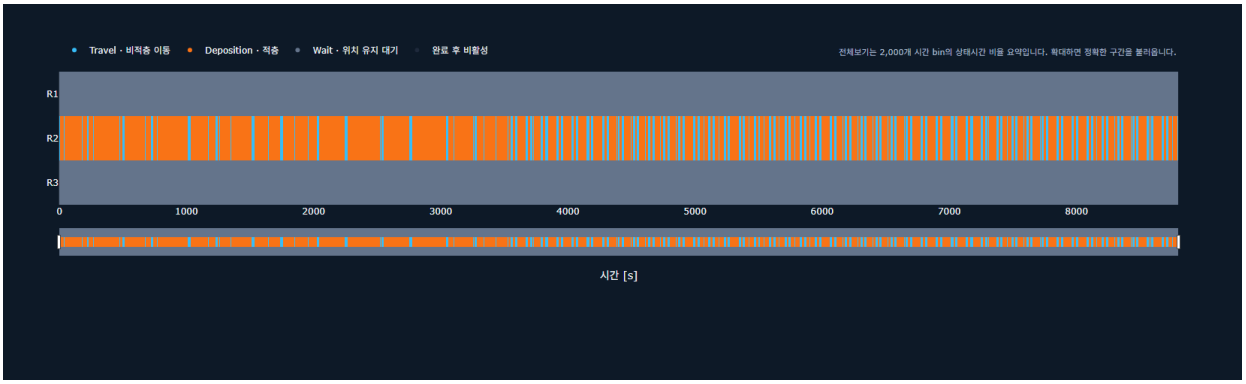
# Minor1 과 Retry1 WAAM Validator 간트 비교

06 Angle frame | Minor1 00:02:12:28 | Retry1 00:02:26:40

Minor1




1.1.1.e retry1



## Algorithm 1. 1.1.1.e retry1 - layer PPO scheduling

```
Input:  STL M, robot configuration C, training set D_tr, validation set D_val
Output: trained PPO policy and collision-checked robot trajectories
01      Sample solid interior points from M; give every 3D sample weight 1
02      Project samples to XY and preserve multiplicity from different Z heights
03      Search rigid XY shifts that fit the workspace and all robot reach regions
04      Assign each projected point to its nearest reachable robot
05      Choose the shift minimizing the standard deviation of the three point shares;
        break ties by maximum share error, travel cost, then shift magnitude
06      Translate the entire STL by this one XY shift, then slice it into layers
07      For each layer, create fixed-spacing hatch segments at the selected angle
08      Split segments at reach-circle boundaries and nearest-base bisectors;
        assign each segment to its nearest reachable robot
09      Policy selects hatch angle (8), each robot start XY (33 x 33),
        each sweep direction (CW/CCW), and one of 6 priority orders
10      Project each requested start onto an owned segment, split it there, and order
        the robot segments by the selected angular sweep
11      for each robot r in the selected priority order do
12          for each deposition task q owned by r do
13              Test approach, deposition, and safe return against reserved trajectories
14              if safe then reserve q for r
15              else keep q pending and continue with the remaining owned tasks
16          Retry pending tasks after any task is accepted; stop if none remain or
            a full sweep accepts no task
17      Transfer only unresolved tasks to the highest-priority robot that can reach them;
        if none qualifies, retain the tasks in the original robot deferred queue
18      Append deferred work after primary routes; delay its home release only if needed
19      Verify every task occurs once, deposition length is preserved, and trajectories
        pass physical collision and trajectory-semantics checks
20      Set layer reward r_l = -T_l; episode return = -sum_l T_l = -makespan
21      Train PPO for 40 updates x 64 steps; keep the lowest validation-mean-makespan
        checkpoint; do not early-stop. Run WAAM Validator for evaluation only
```